

Тестовое задание: To-Do List на Laravel + Vue/Nuxt

Версия для frontend/fullstack-кандидата.

Цель задания

Разработать мини-приложение «Список задач» с авторизацией, CRUD-операциями и клиентской частью на Vue/Nuxt. Задача проверяет умение кандидата собрать рабочий продукт на типичном SPA/API-стеке: Laravel отвечает за данные, авторизацию и бизнес-правила, Nuxt/Vue отвечает за интерфейс и взаимодействие с API.

Обязательный стек

- Backend: Laravel, PHP 8.2+, Eloquent ORM, migrations, seeders, REST API, Form Request validation.
- Frontend: Laravel, Vue 3/Nuxt 3 (4), Composition API, маршруты, middleware/route guard, composables или store для работы с API.
- База данных: SQLite, MySQL или PostgreSQL. Для тестового предпочтительно SQLite.
- Авторизация: Laravel Sanctum или JWT. Выбранный подход нужно кратко описать в README.

Основное задание

Backend: Laravel API

- Реализовать REST API для задач и авторизации.
- Создать миграции, модели, фабрики или сиды для стартового пользователя и тестовых задач.
- Использовать Form Request или аналогичный слой Laravel-валидации для входящих данных.
- Возвращать ошибки в JSON-формате: 401 для неавторизованного запроса, 403 для запрета доступа, 404 для отсутствующей сущности, 422 для ошибок валидации, 500 для непредвиденной ошибки.
- Ограничить редактирование и удаление задач владельцем задачи. Админская роль может быть реализована как дополнительная часть.

Модель задачи

Поле	Тип	Описание
id	integer/uuid	Уникальный идентификатор задачи
user_id	integer/uuid	Владелец задачи
title	string	Обязательный заголовок, 3-255 символов
description	text nullable	Описание задачи
due_date	date nullable	Дедлайн
status	enum	pending, in_progress, completed
created_at / updated_at	datetime	Даты создания/изменения задачи

API-контракт

Метод	Endpoint	Назначение	Доступ
POST	/api/auth/login	Авторизация пользователя, возврат токена или установка cookie-сессии	public
POST	/api/auth/logout	Завершение сессии, инвалидация токена	auth
GET	/api/user	Получение текущего пользователя	auth
GET	/api/tasks	Список задач с сортировкой, фильтрами, поиском и пагинацией	auth
POST	/api/tasks	Создание задачи	auth
GET	/api/tasks/{id}	Получение одной задачи	auth
PUT/ PATCH	/api/tasks/{id}	Редактирование задачи	owner/admin
DELETE	/api/tasks/{id}	Удаление задачи	owner/admin

Frontend: Vue/Nuxt

- Реализовать страницу входа с полями email/password и обработкой ошибок авторизации.
- Хранить состояние авторизации выбранным способом: cookie-сессия Sanctum или Bearer token. При 401 пользователь должен быть перенаправлен на страницу входа.
- Создать страницу списка задач: отображение, сортировка по дедлайну и статусу, фильтрация по статусу.
- Сделать форму добавления задачи: заголовок, описание, дедлайн, статус.

- Добавить редактирование и удаление задач через модальное окно, отдельную форму или inline-режим.
- Показывать состояния загрузки, ошибки API и пустое состояние списка.
- Реализовать базовую валидацию форм на клиенте: обязательный заголовок, корректная дата, допустимый статус.

Дополнительная часть

- Разделение ролей: admin/user. Админ видит все задачи, пользователь только свои.
- Скрывать кнопки редактирования и удаления, если у пользователя нет доступа к действию.
- Поиск по задачам с debounce и синхронизацией с query-параметрами URL.
- Пагинация на backend и frontend.
- Unit/feature-тесты для Laravel API и хотя бы несколько frontend-тестов для критичных сценариев.
- OpenAPI/Swagger-документация или краткое описание API в README.
- Docker Compose для быстрого запуска проекта на проверку.

Формат сдачи

1. Ссылка на репозиторий с исходным кодом.
2. README с командами установки, запуска, миграций, сидов и тестового пользователя.
3. Файл .env.example без секретов и с понятными значениями по умолчанию.
4. Краткое описание выбранного auth-подхода: Sanctum cookie session или Bearer/JWT token.
5. Скриншоты (несколько) или короткое видео работы приложения (не обязательно).

Критерии оценки

Критерий	Что оценивается
Запуск проекта	README, .env.example, миграции, сиды, понятные команды запуска frontend и backend.
Backend	Корректные Laravel-модели, миграции, валидация Form Request, политики доступа и единый JSON-формат ошибок.
Frontend	Структура Nuxt/Vue, работа с API, состояние, маршруты, формы, состояния загрузки и ошибки.
Качество кода	Читаемость, типизация там, где она используется, отсутствие дублирования, аккуратная декомпозиция.
UX	Понятный список задач, редактирование без перезагрузки страницы, пустые состояния, responsive-базовая верстка.

Минимальный результат

Задание считается выполненным, если проект можно запустить по README, пользователь может войти, увидеть список задач, создать задачу, изменить ее статус, отредактировать и удалить задачу, а ошибки API корректно отображаются в интерфейсе.